

Lab 7

Task 1.1

In this task, I prepared mechanical system with one degree of freedom system with matlab using given parameters.

```
% Parameters
M = 1;           % mass (kg)
C = pi;          % damping coefficient
k = 1089 * pi^2; % stiffness

% Natural frequency and damping ratio
wn = sqrt(k/M); % undamped natural frequency (rad/s)
fn = wn / (2*pi); % natural frequency (Hz)
zeta = C / (2 * sqrt(k * M)); % damping ratio

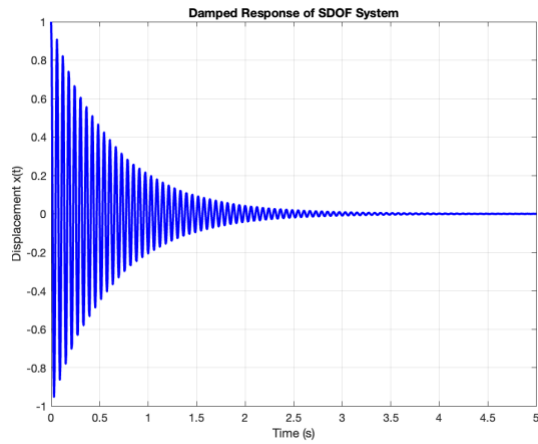
% Display values
fprintf('Natural Frequency (Hz): %.4f\n', fn);
fprintf('Damping Ratio: %.4f\n', zeta); % Time settings
t = linspace(0, 5, 1000); % time vector

% Initial conditions
x0 = 1;          % initial displacement
v0 = 0;          % initial velocity

% Analytical solution for underdamped case
wd = wn * sqrt(1 - zeta^2); % damped natural frequency
A = x0;
B = (v0 + zeta*wn*x0)/wd;
x = exp(-zeta*wn*t) .* (A*cos(wd*t) + B*sin(wd*t));

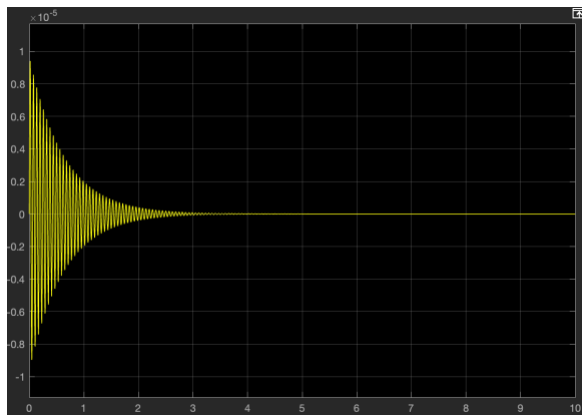
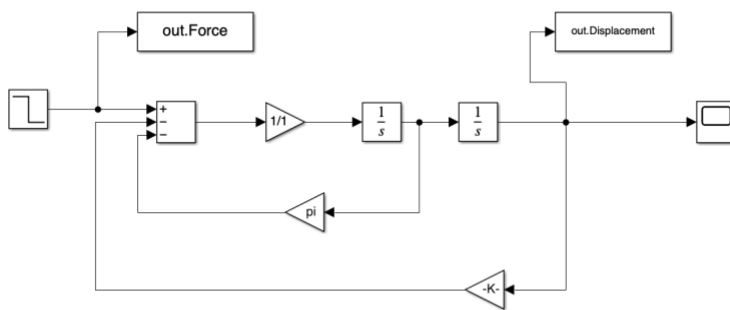
% Plot response
figure;
plot(t, x, 'b', 'LineWidth', 2);
xlabel('Time (s)');
ylabel('Displacement x(t)');
title('Damped Response of SDOF System');
grid on;

% Estimate successive peak ratio x0/x1
[pks, locs] = findpeaks(x, t);
if length(pks) >= 2
    peak_ratio = pks(1) / pks(2);
    fprintf('x0/x1 peak ratio: %.4f\n', peak_ratio);
else
    fprintf('Not enough peaks found to compute x0/x1 ratio.\n');
end
```



Task 1.2

In this task, I prepared mechanical system with one degree of freedom system with Simulink using given parameters and given Simulink instructions.



We got same resultsof plots in different methods for mechanical system with one degree of freedom.

Task 2

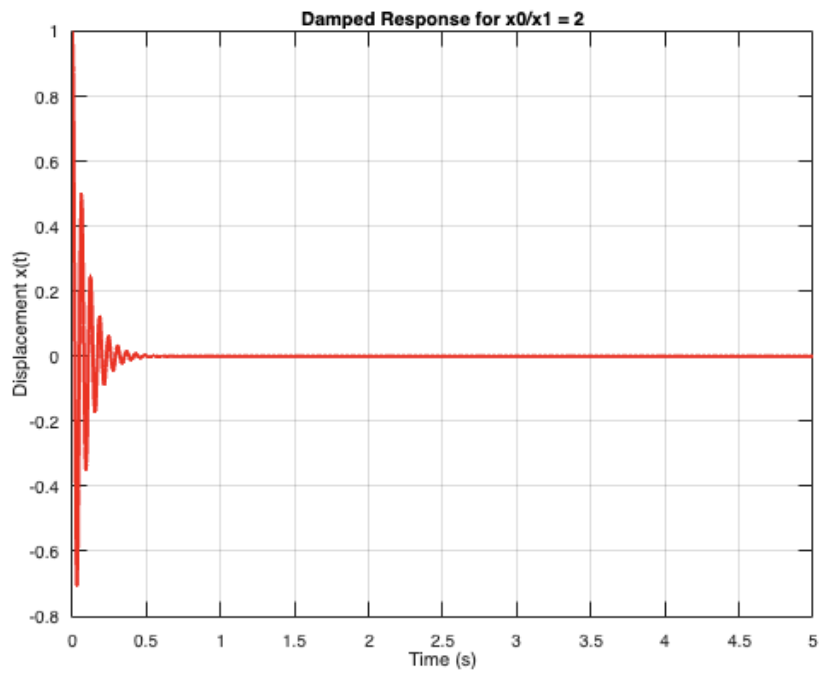
In these exercises for doing $x_0/x_1=2$. I used formulas given in the instruction first page. I adapted them in the code below.

```
M = 1;
k = 1089 * pi^2;
target_ratio = 2;
delta = log(target_ratio);
% Solve for damping ratio using fsolve
zeta_eq = @(zeta) (2*pi*zeta) / sqrt(1 - zeta^2) - delta;
zeta0 = 0.05; % initial guess
zeta = fsolve(zeta_eq, zeta0);
% Calculate corresponding damping coefficient C
C = 2 * zeta * sqrt(k * M);
% Display computed damping parameters
fprintf('Target peak ratio x0/x1 = %.2f\n', target_ratio);
fprintf('Calculated damping ratio  $\zeta$  = %.4f\n', zeta);
fprintf('Calculated damping coefficient C = %.4f\n', C);
% Recalculate natural frequencies
wn = sqrt(k/M);
fn = wn / (2*pi);
wd = wn * sqrt(1 - zeta^2);
% Time vector
t = linspace(0, 5, 1000);
% Initial conditions
x0 = 1;
v0 = 0;
% Damped response
A = x0;
B = (v0 + zeta*wn*x0)/wd;
x = exp(-zeta*wn*t) .* (A*cos(wd*t) + B*sin(wd*t));
% Plot the new response
figure;
plot(t, x, 'r', 'LineWidth', 2);
xlabel('Time (s)');
ylabel('Displacement x(t)');
title('Damped Response for x0/x1 = 2');
grid on;
% Check ratio of first two peaks
[pks, locs] = findpeaks(x, t);
if length(pks) >= 2
    peak_ratio = pks(1) / pks(2);
    fprintf('Simulated peak ratio x0/x1 = %.4f\n', peak_ratio);
```

```

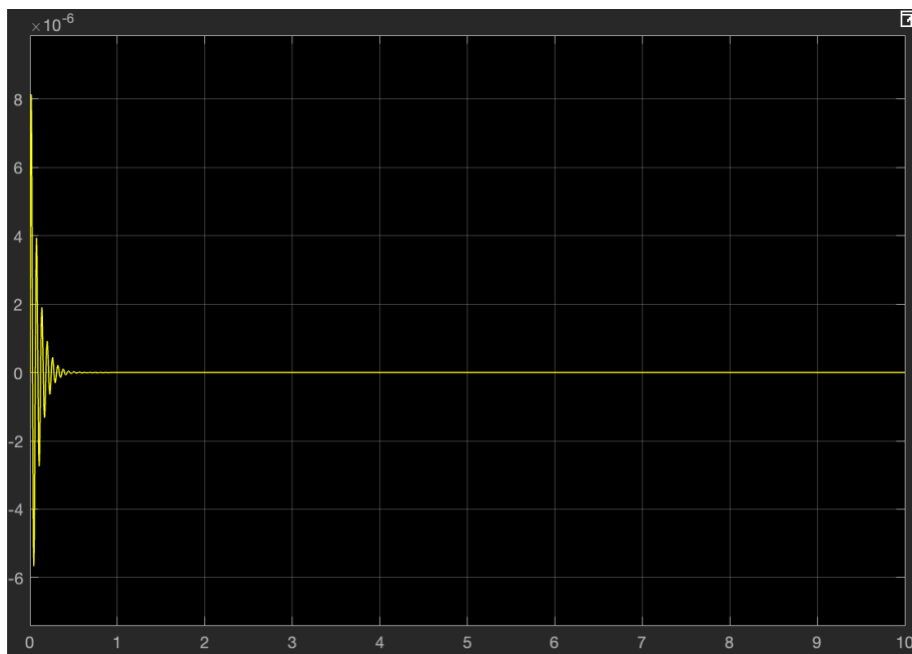
else
    fprintf('Not enough peaks to compute ratio.\n');
end

```



Simulated peak ratio $x_0/x_1 = 2.0278$

For the Simulink, I changed to C to 23.79 in Simulink. Because in the code we found damping ratio as 0.1103 and we implemented that to formula for C .

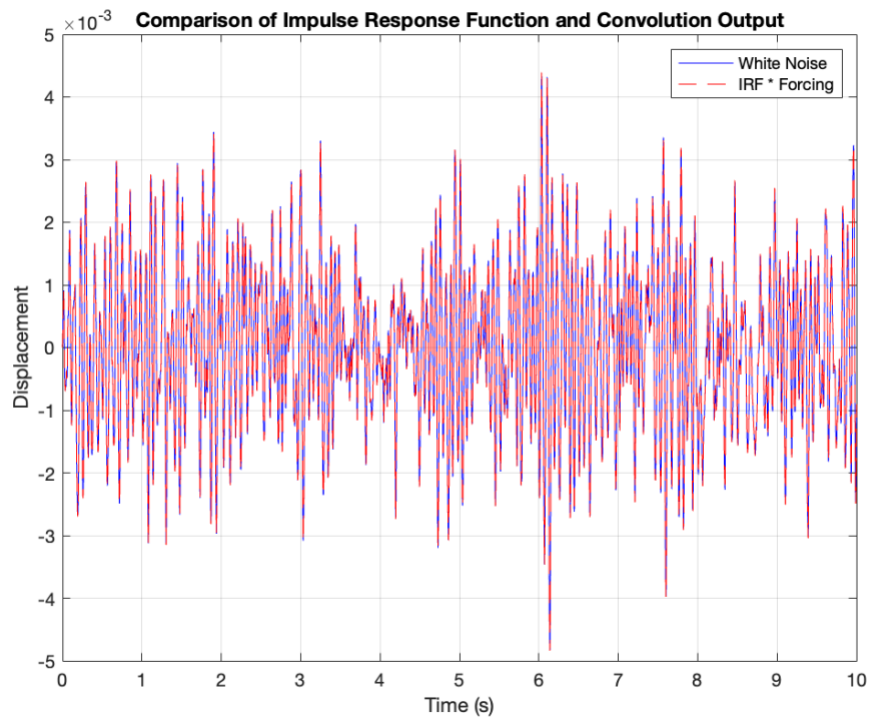


We have same scopes in so we found and used correct values .

Task 3

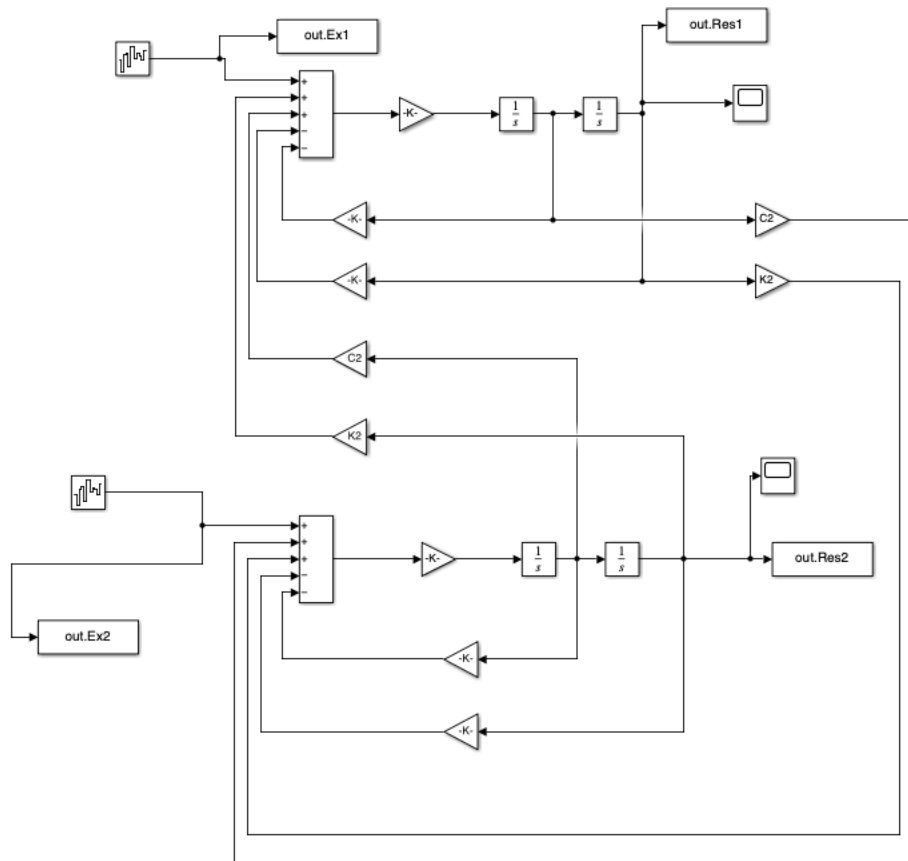
```
IRF = out.Displacement.Data;
forcing_noise = out.Force1.Data;
response_noise = out.Displacement1.Data;
Ts = 0.001;
conv_result = conv(IRF, forcing_noise);
conv_result = conv_result(1:length(response_noise));
t = (0:length(response_noise)-1)*Ts;
figure;
plot(t, response_noise, 'b', 'DisplayName', 'White Noise');
hold on;
plot(t, conv_result, 'r--', 'DisplayName', 'IRF * Forcing');
xlabel('Time (s)');
ylabel('Displacement');
legend;
title('Comparison of Impulse Response Function and Convolution Output');
grid on;
```

This code shows how we can use the impulse response function (IRF) of a system to predict its output for a new input. First, we get the IRF from a step input simulation. Then, we run another simulation with white noise as input and save both the input and output. We use convolution to combine the IRF with the white noise input to get an estimated output. Finally, we plot both the real output and the estimated output to see how well they match. If they are close, it means our IRF is correct.



Task4

In this task, I prepared 2 degree of freedom system as wanted.



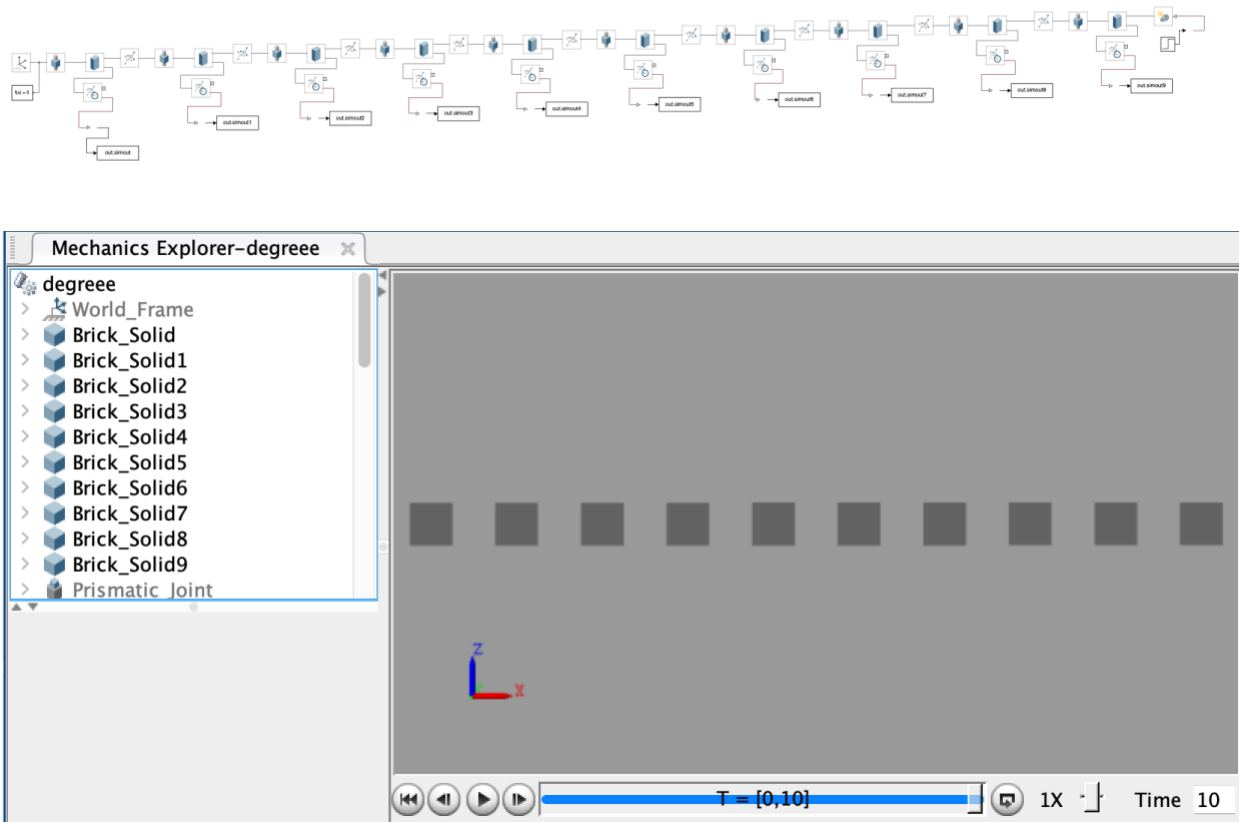
```

M1=1;
M2=1;
K1=10000;
K2=1000000;
K3=10000;
C1=10;
C2=10;
C3=10;

```

Task5

In this task, I prepared 10 degree of freedom system as wanted.



Task6

I simulated with a duration 100 second a sampling frequency of 1000Hz. I used 10th mass for this part. For the 10th mass. It is similar to given spectrum in the instructions.

```
Fs = 1000;  
y = out.simout9;  
L = length(y);  
Y = fft(y);  
P2 = abs(Y/L);  
P1 = P2(1:floor(L/2)+1);  
P1(2:end-1) = 2*P1(2:end-1);  
f = Fs*(0:(L/2))/L;  
semilogy(f, P1)  
xlabel('Frequency (Hz)')  
ylabel('Amplitude (log scale)')  
title('Amplitude-Frequency Spectrum (100 s simulation)')  
grid on  
xlim([0 35])
```

